

Comment l'alignement de
l'architecture logicielle sur les
contraintes matérielles et le tuning
de l'environnement d'exécution
permettent-ils de transformer la
Java Virtual Machine en une
plateforme déterministe et
distribuée pour le trading haute
fréquence ?

Victorien GBAGUIDI

Organisme d'accueil : **Société Générale CIB**
Tuteur Pédagogique : **Khalid BELHAJJAME**
Maître d'apprentissage : **Olivier CHANTEFOIN**

Master 2 MIAGE SITN
Année Universitaire 2025–2026

Table des matières

Thanks	1
Acronyms	1
Introduction	1
1 État de l'art — Le Mur de la Mémoire et les Limites de la JVM	2
1.1 La physique de l'informatique appliquée à la programmation à faible latence	2
1.1.1 Les limites matérielles et les cycles CPU	2
1.1.2 Mesurer la performance	3
1.2 La JVM face au déterminisme financier	4
1.2.1 L'illusion du temps continu : Stop-The-World et Safepoints	5
1.3 Vers une architecture <i>lock-free</i>	5
1.3.1 L'échec de la synchronisation standard : Le coût des verrous	6
1.3.2 Anatomie du Ring Buffer et du <i>pattern "Disruptor"</i>	8
1.3.3 Optimisation de la localité : Padding et Data-Oriented Design	9
1.4 L'OS et le Réseau : Les obstacles à la distribution	9
1.4.1 Configuration et exécution haute performance	9
1.4.2 Du Kernel Stack au Kernel Bypass : Vers le Zero-Copy réseau	10
1.4.3 Optimisation des messages : Simple Binary Encoding (SBE)	12
1.5 Analyse des Solutions Industrielles	12
1.5.1 Agrona et la gestion de la mémoire "Off-Heap"	12
1.5.2 Chronicle Software : Persistance native et Communication Inter-Processus (<i>IPC</i>)	13
2 Structure de la Solution et Implémentation Itérative	15
2.1 Conception générale du <i>matching engine</i>	15
2.1.1 Contexte métier et technique	15
2.1.2 Exigences fonctionnelles du <i>matching engine</i>	16
2.1.3 Exigences non-fonctionnelles du <i>matching engine</i>	16
2.1.4 Modèle de données	17
2.1.5 Architecture du matching engine	18
2.2 <i>Continuous Benchmarking</i> pour le <i>matching engine</i>	19
2.2.1 Contexte général	19
2.2.2 Architecture du pipeline de <i>Continuous Benchmarking</i>	20
2.3 Jalon 0 - L'Order Matching Engine en Programmation Orientée Objets classique et ses inconvénients	22
2.3.1 Analyse de l'OrderBook et de sa structure avec Java Object Layout	22
2.3.2 L'état d'un ordre : la classe Order	25
2.3.3 Le moteur de matching : la classe MatchingEngine	26

2.3.4	Le pipeline producteur-consommateur	30
2.3.5	Mesure de la baseline : méthodologie JMH et interprétation des résultats	34
	Bibliography	37

Table des figures

1	Contention de ressources sur une file d'attente	7
2	Architecture d'un CPU à deux cœurs avec des caches distincts	7
3	Anatomie du Disruptor autour d'un <i>ringbuffer</i> pour une architecture lock-free	8
4	Flux de réception des paquets via le Kernel - Source : Low Latency High-speed Systems - The Kernel Bypass Approach Ijaz Ahmad	11
5	Représentation d'un flux de réception de paquets avec kernel bypass - Source : Low Latency High-speed Systems - The Kernel Bypass Approach Ijaz Ahmad	11
6	Pipeline <i>Continuous Benchmarking</i> du <i>matching engine</i>	21
7	Disposition mémoire de <code>Order</code> obtenue avec Java Object Layout	22

Liste des tableaux

1	Hiérarchie des latences d'accès mémoire en cycles CPU	3
2	Analyse comparative du coût de la synchronisation (en ms) Source : LMAX Disruptor : High performance alternative to bounded queues for exchanging data between concurrent threads	6
3	Durée de transmission par paquet selon la bande passante et le débit - Source Low Latency High-speed Systems - The Kernel Bypass Approach Ijaz Ahmad	10
4	Latences par percentile du chemin critique <code>process()</code> , en ns/op. Extraites de <code>jmh-result.json</code>	36

Introduction

Dans l'écosystème financier contemporain, la vitesse n'est plus un simple avantage compétitif mais la condition essentielle de l'existence des acteurs de marché. Cette course effrénée à la vitesse a propulsé le trading haute fréquence au devant de la scène. Les décisions de vente et d'achat se prennent alors en quelques nanosecondes. Un élément crucial des systèmes financiers modernes est le ***Order Matching Engine*** qui fait correspondre des ordres d'achat à des ordres de vente pour exécuter une transaction. Une priorité selon le prix et le temps est alors effectuée. L'ordre d'achat le plus élevé pour un ordre de vente donné est priorisé, et à prix égal, l'ordre ayant été effectué en premier a la priorité. On voit alors de manière évidente les enjeux liés à la vitesse d'exécution.

Dans ce cadre, le C++ s'est imposé comme langage de programmation de référence pour ses possibilités d'optimisation bas niveau (gestion manuelle de la mémoire et interaction directe avec le matériel). À l'inverse, Java a mis du temps à trouver sa place au sein de l'univers de l'ultra faible latence. Son modèle de gestion automatique de la mémoire pilotée par le garbage collector crée des pauses imprévisibles qui augmentent fortement la latence dans les pires cas, une situation inacceptable pour les marchés financiers.

Une nouvelle philosophie propulsée par Martin Thompson, co-fondateur de LMAX, la mechanical sympathy propose que les performances d'un programme dépendent de la capacité du programmeur à le développer en s'alignant sur les contraintes matérielles du CPU et de la mémoire. Je vais dans ce mémoire explorer comment, en appliquant les concepts de mechanical sympathy, on peut transformer la JVM en environnement déterministe et à faible latence pour tirer partie du riche écosystème de Java tout en rivalisant avec les performances du C++ au niveau de la latence. La problématique sera donc : **Comment l'alignement de l'architecture logicielle sur les contraintes matérielles et le tuning de l'environnement d'exécution permettent-ils de transformer la Java Virtual Machine en une plateforme déterministe et distribuée pour le trading haute fréquence ?**

Pour répondre à cette problématique, mon plan s'articulera en deux parties :

- Un état de l'art : j'analyserai la hiérarchie des caches CPU, les obstacles internes de la JVM (Just In Time, Safepoints, Stop-the-World, ...) et nous étudierons les solutions industrielles telles que LMAX Disruptor, Aeron et Chronicle Software
- Une implémentation incrémentale d'un *order matching engine* : je détaillerai la construction de ma solution à travers dix jalons qui montrent l'évolution de la performance des différentes versions du matching engine en partant d'une base qui s'appuie sur des principes de programmation orientée objet puis nous appliqueront une architecture Zéro Garbage Collector et Data Oriented ainsi que des optimisations systèmes qui permet-

tront de montrer les gains apportés par l'application de la *mechanical sympathy* à notre problématique.

Mon intérêt pour cette problématique prend racine dans une passion profonde pour l'ingénierie logicielle. Je suis fermement convaincu que l'excellence d'un ingénieur réside dans sa maîtrise de la *mechanical sympathy*. Au-delà de cet attrait personnel, ce mémoire s'inscrit dans un enjeu industriel majeur. Dans des environnements de finance de marché où les capitaux exposés au risque se chiffrent en millions d'euros à chaque microseconde, l'optimisation n'est plus une option, mais une nécessité économique vitale. Ce projet est pour moi l'opportunité de démontrer que la rigueur de l'implémentation est un levier majeur de création de valeur financière.

1 État de l'art — Le Mur de la Mémoire et les Limites de la JVM

1.1 La physique de l'informatique appliquée à la programmation à faible latence

1.1.1 Les limites matérielles et les cycles CPU

Structure et type de RAM - Drepper, 2007 : Privilégier la SRAM (Static RAM), support physique des mémoires caches, à la DRAM (Dynamic RAM) permet de bénéficier d'un déterminisme temporel intrinsèque à sa structure à bascule qui s'affranchit des cycles de rafraîchissement de la DRAM. À l'inverse, cette dernière introduit un indéterminisme structurel : la nécessité de régénérer périodiquement la charge de ses condensateurs crée des fenêtres d'indisponibilité asynchrones, générant de la gigue sur le chemin critique du *matching engine*.

Dès lors, l'enjeu n'est plus seulement de minimiser la latence mais aussi de maximiser la localité spatiale et temporelle des structures de données du *matching engine* pour optimiser le hit rate (indicateur mesurant le taux de données récupérées depuis le cache sans devoir solliciter la DRAM). Nous limitons ainsi les accès à la mémoire principale, garantissant une exécution déterministe minimisant les cycles d'attente.

Les opérations en cache : La performance d'un *matching engine* dépend de l'accès aux données. Le tableau 1 illustre la hiérarchie des latences selon les chiffres de Drepper. Supposons un processeur qui a un cycle d'horloge de 0.25ns (i.e une fréquence de 4GHz), l'écart de performance entre un cache L1 (1ns) et la mémoire principale (60ns) met en évidence l'impact critique des *caches misses*. Une absence des informations au sein du cache impose un temps d'attente de 240 cycles durant lesquels le processeur demeure inactif.

Destination	Latence (Cycles CPU)
Registres	≤ 1
Cache L1d	~ 3
Cache L2	~ 14
Mémoire principale	~ 240

TABLE 1 – Hiérarchie des latences d'accès mémoire en cycles CPU

Cette disparité impose deux contraintes majeures pour le développement du *matching engine* :

- la densité informationnelle : l'usage d'objets Java standards introduit une surcharge mémoire via les en-têtes d'objets. Pour optimiser la probabilité de résidence des ordres dans la *cache line* (unité logique de 64 octets), des structures de données contiguës basées sur des types primitifs seront privilégiées.
- l'exploitation du *hardware prefetcher* : ce composant anticipe les besoins du processeur en chargeant les données futures basées sur la localité spatiale. Une organisation mémoire éparpillée (inhérente aux listes chaînées par exemple) neutralise ce mécanisme, forçant des accès répétés à la DRAM qui dégradent fortement les performances du *matching engine*

La compréhension de ces micro-mécanismes matériels révèle la volatilité extrême de la performance à l'échelle de la nanoseconde. Dès lors que l'exécution n'est plus linéaire mais rythmée par les cycles d'attente du processeur (*CPU stalls*), les outils de métrologie conventionnels deviennent insuffisants. Afin de valider l'efficacité de nos optimisations sur le chemin critique, il convient d'adopter une méthodologie de mesure rigoureuse, capable d'isoler les biais logiciels et de quantifier précisément la gigue du système.

1.1.2 Mesurer la performance

Pour mesurer la performance d'un programme, nous avons deux métriques fondamentales : le **throughput** (débit) qui désigne le volume d'opérations traitées par période de temps et la **latence** qui mesure le délai d'exécution d'une instruction isolée. Dans le cadre de ce mémoire, nous nous focaliserons sur la latence. En effet, la rentabilité d'un *matching engine* dépend de sa capacité à réagir à un signal de marché plus vite que ses concurrents.

Nous privilégierons le **Wall Clock Time** (temps réel écoulé) car il englobe l'intégralité des délais subis par le programme : accès mémoire, opérations d'entrée/sortie et la surcharge système (OS overhead). À l'inverse, le **CPU Time** sera considéré comme incomplet car il ignore les phases d'attente (comme les pauses du Garbage Collector ou les interruptions réseau) qui sont pourtant critiques en production.

De plus, nous rejeterons la latence moyenne au profit de l'analyse des percentiles et plus particulièrement la latence P99.995 qui indique que 99.995% des mesures auront une latence inférieure ou égale à la valeur seuil. En finance de marché, l'usage de la moyenne arithmétique serait une erreur méthodologique majeure car elle masque les valeurs aberrantes capables d'engendrer des pertes de plusieurs millions d'euros. Ce choix du P99.995 s'appuie sur les démonstrations de Gil Tene concernant les probabilités de subir les pires latences. Tene démontre que pour une session utilisateur standard, par exemple le chargement de cinq pages web impliquant chacune quarante ressources, la probabilité d'être exposé à au moins une occurrence dépassant le seuil P95 grimpe à 99,007%. Même au seuil P99.9, environ 18% des utilisateurs subissent encore ces pics de latence. Dès lors, pour garantir que plus de 99% des sessions de trading s'exécutent sans rencontrer de gigue majeure, il est mathématiquement impératif de viser le percentile P99.995. Cette métrique devient notre étalon pour quantifier le gain de performance réel entre chaque jalon de développement. Nous prendrons également soin de systématiquement mettre en évidence la latence maximale pour toujours avoir une vision sur le pire des cas et la limiter.

Enfin pour respecter la reproductibilité de nos mesures (critère essentiel d'un benchmark selon Hennessy et Patterson), nous utiliserons le framework ***Java Microbenchmark Harness (JMH)***. Cet outil, référence de l'industrie, permet d'isoler les biais de la JVM tels que l'élimination de code mort et de gérer le *warmup* (phase de compilation du programme en code natif au système). Il garantit que les gains observés entre nos différents jalons sont statistiquement significatifs. L'usage de JMH est d'autant plus critique qu'il permet de minimiser les biais d'omission coordonnée (*coordinated omission*). JMH inclut les pauses du système, là où un benchmark plus naïf ignorerait ces périodes d'inactivité.

1.2 La JVM face au déterminisme financier

L'efficacité du *matching engine* sur le chemin critique est également conditionnée par la gestion du cycle de vie des objets. L'utilisation de la mémoire conventionnelle de la JVM (*Heap*) expose le système aux pauses *Stop-The-World* (STW) du *Garbage Collector*. Ces interruptions, nécessaires à la récupération des objets non-référencés, introduisent un indéterminisme structurel majeur et une gigue imprévisible, incompatibles avec les enjeux de faible latence. L'industrie privilégie des structures de données *off-heap* ou, à défaut, des agrégats contigus de types primitifs permettant de réduire drastiquement la densité du graphe d'objets et la pression sur le collecteur.

De surcroît, cet abandon de la sémantique objet permet de s'affranchir du phénomène de *pointer chasing*. Dans une structure d'objets liés (telle qu'une liste chaînée), l'adresse de l'élément suivant n'est connue qu'au chargement complet de l'élément courant. Cela empêche toute anticipation et provoque des calages systématiques du pipeline (*stalls*). À l'inverse, un agencement contigu en mémoire permet au *hardware prefetcher* d'identifier un motif d'accès linéaire. Par

simple incrémentation de pointeur, le CPU peut alors anticiper les requêtes vers la DRAM et masquer la latence d'accès en pré-chargeant les données critiques dans les caches supérieurs.

1.2.1 L'illusion du temps continu : Stop-The-World et Safepoints

Dans ses travaux (*Understanding Java Garbage Collection*), Gil Tene démontre que l'intégralité des implémentations de collecteurs, y compris les versions dites « précises », introduisent des pauses inévitables. Le cycle de vie d'un collecteur repose sur trois opérations critiques :

- l'identification des objets vivants (objets utilisés par le programme)
- la récupération des ressources
- la relocalisation des données

Ces opérations peuvent être effectuées successivement (*Mark/Sweep/Compact Collector*) ou en une fois (*Copy Collector*). Pour les deux collecteurs, l'opération de relocalisation des données (par compactage ou par copie vers un nouvel espace mémoire) est la plus coûteuse. Si certaines phases peuvent être exécutées de manière concurrente, la phase de mouvement des objets (*Compaction / Evacuation*) nécessite de mettre à jour l'intégralité des références, imposant un arrêt complet de l'application : le *Stop The World (STW)*.

Selon les métriques de Tene, ces pauses peuvent atteindre une seconde par gigaoctet de tas (*heap*), créant une gigue massive totalement décorrélée de la logique métier. À l'échelle de la basse latence, une pause d'une seconde crée une gigue seize millions de fois supérieure à un accès DRAM, une éternité informatique qui rend la gestion automatique de la mémoire incompatible avec les exigences du trading haute fréquence. D'après un sondage parmi les participants à la conférence de Tene le tas d'une application oscille généralement entre 1Go et 4Go.

Cette réalité physique impose à l'industrie du trading haute fréquence une stratégie de contournement radicale : la migration des données critiques vers des structures hors-tas (*off-heap*). En soustrayant le *matching engine* à la juridiction du *Garbage Collector*, l'objectif n'est pas seulement de réduire la durée des collectes, mais d'éliminer purement et simplement le risque d'indisponibilité du *matching engine* sur le chemin critique.

1.3 Vers une architecture *lock-free*

La programmation concurrente conventionnelle repose sur l'exclusion mutuelle (*mutex*) pour garantir l'intégrité des données. Ce modèle impose aux *threads* un accès séquentiel aux ressources via des verrous (*locks*), forçant les unités d'exécution concurrentes à l'inactivité. Comme l'illustre le tableau 2, l'impact de la synchronisation est dévastateur : pour cinq cent millions d'incrémentations d'un compteur, le passage d'un *thread* unique sans contrainte à une gestion avec

verrous entre deux *threads* fait exploser le temps d'exécution de 300 ms à 224 000 ms, soit un facteur supérieur à 700. Dès lors, pour atteindre des latences de l'ordre de la nanoseconde, le passage à une architecture *lock-free* devient impératif. L'enjeu technique consiste à maintenir la cohérence des données sans interrompre le flux du processeur. Pour répondre à cette problématique, l'équipe de LMAX a introduit le *Disruptor* : un *pattern* basé sur une structure de données circulaire offrant une alternative haute performance aux files d'attente synchronisées traditionnelles.

Method	Time (ms)
Single thread	300
Single thread with lock	10 000
Two threads with lock	224 000
Single thread with CAS	5 700
Two threads with CAS	30 000
Single thread with volatile write	4 700

TABLE 2 – Analyse comparative du coût de la synchronisation (en ms)
Source : LMAX Disruptor : High performance alternative to bounded queues for exchanging data between concurrent threads

1.3.1 L'échec de la synchronisation standard : Le coût des verrous

Analyse de la contention - LMAX Disruptor Les travaux de l'équipe LMAX mettent en lumière une inefficacité structurelle des files d'attente (*queues*) classiques dans un modèle producteur-consommateur. Lorsque la vitesse de consommation excède celle de la production, la taille de la file tend vers zéro, provoquant une contention systématique sur les mêmes adresses mémoires (tête et queue de file). Ce phénomène impose l'usage de verrous, dont l'arbitrage par le noyau système génère une latence paradoxalement supérieure à celle d'un modèle strictement séquentiel.

En effet, l'utilisation d'un verrou délègue la gestion de la concurrence au système d'exploitation. Ce dernier est alors libre d'arbitrer l'exécution des threads, entraînant des changements de contexte coûteux. L'OS peut notamment suspendre des threads critiques au profit d'autres processus, provoquant une "pollution" des caches : les données utiles sont évincées pour laisser place à de nouvelles instructions, forçant les threads du système de trading à les recharger depuis la mémoire RAM. Cette rupture de la localité des données génère une instabilité de la latence incompatible avec les exigences de haute performance.

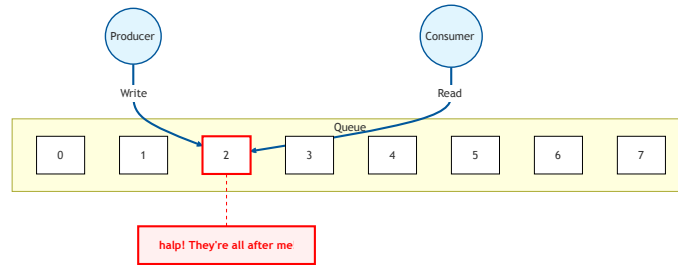


FIGURE 1 – Contention de ressources sur une file d’attente

Par ailleurs, l’architecture des processeurs modernes complexifie le partage de données : chaque cœur dispose de ses propres caches L1 et L2. Lorsqu’un thread modifie une donnée partagée, les caches des autres cœurs ne sont plus à jour, déclenchant des mécanismes de cohérence coûteux qui dégradent les performances. Pour s’affranchir de ces limites matérielles et logicielles, LMAX a conçu le *Disruptor*, une structure de données optimisée pour la concurrence.

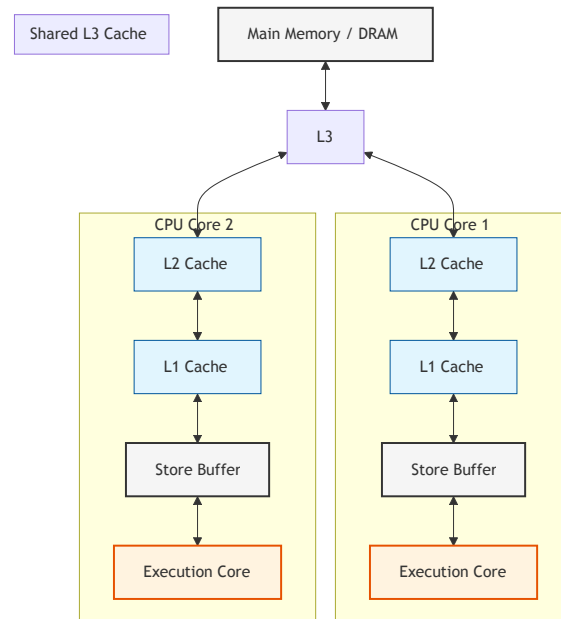


FIGURE 2 – Architecture d’un CPU à deux cœurs avec des caches distincts

1.3.2 Anatomie du Ring Buffer et du *pattern* "Disruptor"

Le principe du Single-Writer et des Memory Barriers - LMAX Disruptor : Afin de pallier les limitations induites par les verrous, l'équipe LMAX a conçu le *Disruptor*, une architecture dont le cœur est un *ring buffer* (tampon circulaire) implémenté sous forme de tableau contigu dont la taille est une puissance de 2, ce qui permet de substituer l'opérateur modulo par un masque binaire pour gérer l'indexation circulaire. Ce modèle repose sur une séparation stricte des responsabilités de gestion d'index. Dans une configuration simple, le producteur dispose de l'exclusivité d'écriture dans le tampon et incrémente son propre compteur de séquence pour signaler la disponibilité de nouvelles données. De son côté, le consommateur accède aux données en lecture seule sans modifier l'état interne de la structure partagée. Il maintient son propre curseur qu'il compare, via une barrière de lecture, à celui du producteur ; cette vérification lui permet de déterminer instantanément le volume de données disponibles. Réciproquement, le producteur surveille la progression du consommateur afin de s'assurer qu'il ne boucle pas sur l'anneau au point d'écraser des événements non encore traités. Cette coordination asymétrique permet une communication inter-threads totalement libre de verrous (*lock-free*).

De son côté, le consommateur accède aux données en lecture seule sans modifier l'état interne de la structure partagée. Il maintient son propre curseur qu'il compare, via une barrière de lecture, à celui du producteur ; cette vérification lui permet de déterminer instantanément le volume de données disponibles. Réciproquement, le producteur surveille la progression du consommateur afin de s'assurer qu'il ne boucle pas sur l'anneau au point d'écraser des événements non encore traités. Cette coordination asymétrique permet une communication inter-threads totalement libre de verrous (*lock-free*).

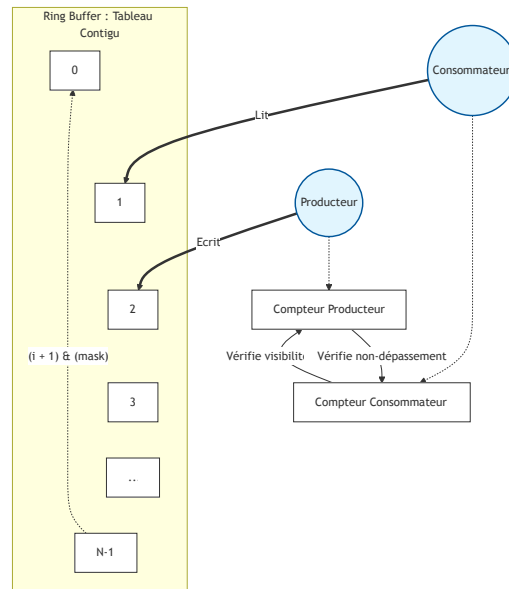


FIGURE 3 – Anatomie du Disruptor autour d'un *ringbuffer* pour une architecture lock-free

1.3.3 Optimisation de la localité : Padding et Data-Oriented Design

Élimination du *False Sharing* Le *false sharing* survient lorsque plusieurs *threads*, s'exécutant sur des cœurs distincts, accèdent à des variables différentes mais situées sur la même ligne de cache (généralement 64 octets). Bien que les variables soient logiquement indépendantes, toute modification de l'une entraîne l'invalidation de la ligne de cache entière pour les autres cœurs, provoquant une contention invisible et coûteuse.

Pour neutraliser ce phénomène, le *Disruptor* emploie la technique du *padding* : elle consiste à entourer les variables critiques (comme les compteurs de séquence) de variables factices afin d'isoler chaque donnée sur sa propre ligne de cache. Cette isolation garantit que le producteur et le consommateur travaillent sur des segments matériels distincts, préservant ainsi la localité des données et maximisant l'efficacité des caches L1 et L2.

1.4 L'OS et le Réseau : Les obstacles à la distribution

Au-delà de l'optimisation algorithmique, l'obtention d'une latence faible et déterministe se heurte à un obstacle externe : l'imprévisibilité du système d'exploitation. En effet, l'ordonnanceur peut suspendre arbitrairement les *threads* critiques du *matching engine* au profit de tâches système. Ces changements de contexte (*context switches*) imposent un cycle coûteux : la mise en pause de l'unité d'exécution, l'éviction des données courantes du cache au profit de celles du nouveau processus, puis, lors de la reprise, le rechargement nécessaire des caches CPU. Ce mécanisme de "va-et-vient" détruit la localité des données et accroît considérablement la gigue (*jitter*) du programme.

Parallèlement au logiciel, le matériel intervient via les requêtes d'interruption (*Interrupt ReQuests* ou *IRQs*). Ces signaux forcent le processeur à suspendre instantanément le flux applicatif pour traiter des événements asynchrones, tels que l'arrivée de paquets réseau ou des accès disque, générant des pics de latence imprévisibles. La recherche d'une performance de l'ordre de la nanoseconde exige donc une stratégie d'isolation totale, visant à soustraire le moteur de trading à l'influence de l'ordonnanceur et des bruits système.

1.4.1 Configuration et exécution haute performance

Afin de résoudre les context switches imposés par l'ordonnanceur, une technique consiste à isoler chaque *thread* de l'application sur un cœur, c'est le *CPU pinning*. En veillant à ce qu'un seul *thread* soit présent par cœur, le CPU ne déclenche pas de changement de contexte. On veillera alors dans le chemin critique à ne jamais créer plus de *threads* que de cœurs CPU. Imaginons un CPU possédant huit cœurs, le *matching engine* possédera sept *threads*, afin de garantir que les sept *threads* critiques du moteur de *matching* ne subissent aucune préemption, l'isolation des cœurs est configurée au niveau du chargeur d'amorçage via la directive `isolcpus` au sein du fichier `/etc/default/grub` :

```
GRUB_CMDLINE_LINUX_DEFAULT="isolcpus=1,2,3,4,5,6,7 nohz_full=1,2,3,4,5,6,7"
```

Listing 1 – Configuration de l’isolation des cœurs dans GRUB

Note : Le huitième cœur est réservé à l’OS.

Une fois les ressources réservées, l’exécution du programme Java est réalisée en forçant l’affinité matérielle avec l’utilitaire `taskset`. Cette commande assure que la machine virtuelle Java (JVM) exploite exclusivement les segments de calcul isolés :

```
taskset -c 1-8 java -XX:+UseNUMA -cp . MonMatchingEngine
```

Listing 2 – Exécution avec affinité CPU et optimisation NUMA

L’option `-XX:+UseNUMA` complète ce dispositif en optimisant l’allocation mémoire selon la topologie physique de la machine, minimisant ainsi les cycles d’attente lors des accès à la mémoire vive (RAM).

1.4.2 Du Kernel Stack au Kernel Bypass : Vers le Zero-Copy réseau

Les limites de la pile réseau standard Dans les systèmes de trading à haute fréquence, la traversée de la pile réseau représente la partie prépondérante de la latence de bout en bout, dépassant souvent le temps d’exécution de la logique métier. Comme l’illustre le tableau 3, le délai de transmission pour un paquet standard passe de 120 000 ns en 100Base-T à seulement 1 200 ns en 10GBase-T. À ce niveau de performance, le coût logiciel devient prohibitif : la pile réseau standard du noyau Linux impose une latence de traversée de l’ordre de plusieurs microsecondes. Dès lors, pour un message de 256 octets, le temps passé par le CPU à copier les données entre les domaines mémoire (du *Ring Buffer* de la NIC vers le *Kernel Buffer*, puis vers le *User Space*) excède mathématiquement le temps de traitement d’un ordre de bourse, souvent inférieur à 500 ns.

Bande Passante	Débit	Durée de transmission par paquet (ns)
100Base-T	100 Mbps	120 000 ns
1000Base-T (1G)	1 Gbps	12 000 ns
10GBase-T	10 Gbps	1 200 ns
100GBase-T	100 Gbps	120 ns
400GBase-T	400 Gbps	30 ns

TABLE 3 – Durée de transmission par paquet selon la bande passante et le débit - Source Low Latency High-speed Systems - The Kernel Bypass Approach Ijaz Ahmad

Le transit d'un paquet du fil vers l'application subit une cascade d'indirections. À la réception, la carte réseau (NIC) génère une interruption matérielle (IRQ), forçant le processeur à suspendre le *thread* applicatif pour exécuter le gestionnaire d'interruption du noyau. Ce mécanisme induit un double coût :

- *Cache Pollution* : le changement de contexte vers le mode noyau évince les lignes de cache dédiées à la logique métier, provoquant un besoin de les recharger à chaque retour au *thread* métier
- *Data Copying* : le paquet est copié du tampon de la NIC vers le *kernel space*

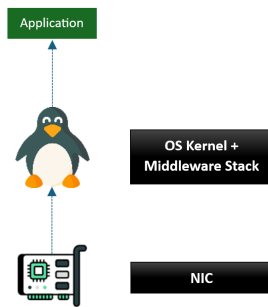


FIGURE 4 – Flux de réception des paquets via le Kernel - Source : Low Latency High-speed Systems - The Kernel Bypass Approach Ijaz Ahmad

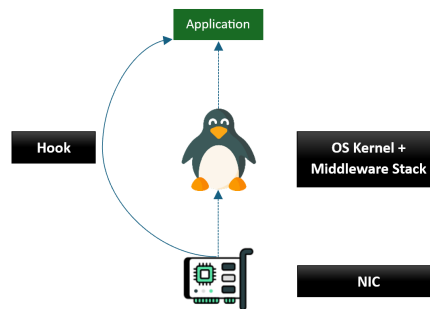


FIGURE 5 – Représentation d'un flux de réception de paquets avec kernel bypass - Source : Low Latency High-speed Systems - The Kernel Bypass Approach Ijaz Ahmad

La révolution du Kernel Bypass et le modèle Zero-Copy : Le *Kernel Bypass* (via DPDK ou des piles propriétaires comme Solarflare Onload) court-

circuite intégralement la couche logicielle de l'OS. La carte réseau écrit les données directement dans une zone mémoire partagée, accessible par l'application en mode utilisateur. L'impact de cette approche est triple :

- Le thread dédié au réseau est épinglé sur un cœur CPU isolé et scrute en permanence les descripteurs de réception de la NIC. On élimine ainsi 100% des interruptions et des changements de contexte.
- L'application gère elle-même les couches de transport (ex : pile TCP/UDP en espace utilisateur), ce qui permet de pré-allouer des buffers et d'éviter toute allocation dynamique sur le chemin critique.
- En alignant ces buffers sur les frontières des lignes de cache (64 octets), le *matching engine* peut consommer le message dès sa réception sans aucun cycle d'horloge gâché par une copie intermédiaire.

Cette approche transforme l'interface I/O réseau en un simple accès mémoire, réduisant la gigue logicielle.

1.4.3 Optimisation des messages : Simple Binary Encoding (SBE)

Le passage d'un protocole textuel (FIX, JSON) ou binaire à structure variable (Google Protocol Buffers) vers SBE répond à une exigence de déterminisme à la nanoseconde.

- **Élimination du coût de parsing et de l'allocation :** La transformation de flux ASCII en objets Java (ex : `String` ou `Double`) est une hérésie en basse latence. Elle déclenche des allocations sur le *heap*, augmentant la pression sur le *Garbage Collector* et générant des pauses *Stop-The-World*. SBE adopte une approche *Allocation-Free* : les données sont lues directement depuis des *DirectBuffers* (souvent mappés via *Agrona* que nous présenterons ultérieurement), sans instanciation d'objets intermédiaires.
- **Philosophie *Copy-Free* et accès séquentiel :** Le design de SBE repose sur le principe de l'accès *streaming*. Contrairement aux protocoles nécessitant un *backtracking* ou des indirections pour accéder aux champs variables, SBE impose que les champs à longueur variable (comme les chaînes de caractères) soient placés en fin de message. Cette contrainte garantit que le processeur accède à la mémoire de manière strictement séquentielle. Cette linéarité est optimale pour le **L1 Data Prefetcher**. Le CPU anticipe le chargement des prochaines *cache lines* (64 octets) dans le cache L1 avant même que le code ne les sollicite, évitant ainsi des cycles d'attente (*stalls*) de pipeline.

1.5 Analyse des Solutions Industrielles

1.5.1 Agrona et la gestion de la mémoire "Off-Heap"

Optimisation des structures de données via Agrona : Pour pallier l'inefficience des collections de la JDK standard, nous intégrons la bibliothèque

Agrona. Les collections `java.util` souffrent de deux tares rédhibitoires pour la très basse latence : l'impossibilité de stocker des types primitifs (induisant un *autoboxing* massif et une pression sur le *Garbage Collector*) et la dispersion en mémoire des données entraînant un phénomène de *pointer chasing*.

Chaque objet Java possède un en-tête (*Object Header*) de 16 octets, ajoutant une surcharge informationnelle et force le processeur à suivre des références vers des zones mémoires non contiguës. Les collections d'*Agrona* reposent sur des structures de données stockées dans des tableaux de primitifs contigus. Cette architecture favorise la localité spatiale : en plaçant les clés et les valeurs (dans le cadre des *maps* et des *sets*) de manière adjacente, *Agrona* maximise la probabilité qu'un couple de données réside dans une même *cache line* de 64 octets. Ce design permet au **Hardware Prefetcher** du CPU d'anticiper les accès mémoire et de charger les données dans le cache L1 avant même leur sollicitation, masquant ainsi la latence de la DRAM.

Communication inter-threads et Ring Buffers : Pour la communication *inter-threads* nous utilisons le `OneToOneRingBuffer` d'*Agrona*. Contrairement aux files d'attente bloquantes de la JDK, cette structure est *lock-free* et *wait-free* pour le producteur. Conformément au *pattern Disruptor*, nous aurons un producteur unique ayant les accès en écriture et des consommateurs qui feront des appels au `OneToOneRingBuffer` pour les accès en lecture.

1.5.2 Chronicle Software : Persistance native et Communication Inter-Processus (IPC)

Pour un *matching engine*, la latence réseau entre la réception d'un ordre et son enregistrement est un facteur de compétitivité critique. L'approche de *Chronicle Software* est de fusionner la communication entre les processus et la sauvegarde des données en une seule opération ultra-rapide basée sur la mémoire partagée.

Le fichier comme vecteur de communication et de persistance : Dans une architecture classique, un ordre doit être envoyé via un *socket* réseau puis sauvegardé dans une base de données (disque). *Chronicle* supprime cette double étape en utilisant les fichiers mappés en mémoire (*Memory Mapped Files*). Le fichier sur disque est projeté directement dans la RAM. Écrire un message pour le transmettre à un autre processus revient donc à l'écrire directement dans le fichier en RAM qui sera directement visible.

L'extrait suivant illustre comment l'écriture devient à la fois un acte de communication et un acte d'archivage :

```
// 1. Ouverture du fichier journal (qui servira de canal de
    communication)
FileChannel fc = new CleaningRandomAccessFile(fileName, "rw"
    ).getChannel();
```

```
// 2. Mappage : le fichier devient une zone de RAM partagée
//     entre les processus
long address = OS.map(fc, MapMode.READ_WRITE, 0, 64 << 10);

// 3. Persistance instantanée : l'ordre est écrit en RAM,
//     mais déjà lié au disque
OS.memory().writeLong(address + 1024L, 0x1234567890ABCDEFL);

// 4. L'ordre est persiste et disponible pour les autres
//     processus sans latence réseau
```

Listing 3 – Ecriture Off-Heap persistante avec Chronicle-Core

Comme la *Gateway* et le *Matching Engine* regardent le même fichier projeté en RAM, l'ordre n'est jamais déplacé. Sa présence dans le fichier suffit à le rendre "public" pour les autres composants. Chaque octet écrit est virtuellement déjà sur le disque. En cas de coupure de courant, le noyau Linux garantit que les données présentes dans cette RAM (le *Page Cache*) sont les dernières enregistrées, permettant une reconstruction immédiate du carnet d'ordres au redémarrage.

S'affranchir du *Garbage Collector* et du disque : La force de Chronicle est de rendre la persistance "invisible" pour le processeur. Le *matching engine* ne subit jamais de ralentissement dû aux accès disques ou aux nettoyages de mémoire. L'écriture se fait à la vitesse de la RAM (nanosecondes). C'est le système d'exploitation qui, de manière asynchrone et en arrière-plan, synchronise physiquement les données sur le support disque. Le thread critique du moteur de matching ne bloque jamais pour attendre le disque. En utilisant la mémoire hors-tas (*Off-Heap*), Chronicle ne crée aucun objet Java pour transporter l'ordre. On évite ainsi l'indéterminisme du *Garbage Collector*.

```
Memory memory = OS.memory();
long address = memory.allocate(1024);
try {
    // Modification atomique (CAS) : coordination sans verrou (
    //     lock-free)
    boolean swapped = memory.compareAndSwapInt(address, 1, 2);
} finally {
    // Libération manuelle : aucun nettoyage automatique
    //     imprévisible
    memory.freeMemory(address, 1024);
}
```

Listing 4 – Gestion déterministe et atomique

L'usage de `compareAndSwapInt` (CAS) permet à plusieurs threads de lire et modifier le journal d'ordres simultanément sans jamais se bloquer mutuellement. Le pipeline du processeur reste saturé d'instructions utiles, maximisant le débit

d'ordres tout en garantissant que chaque transaction est enregistré sur disque sans aucun sacrifice de vitesse.

2 Structure de la Solution et Implémentation Itérative

Un *matching engine* suit la logique du *price-time priority* :

- les ordres d'achat les plus élevés sont priorisés par rapport à ceux plus faibles car ils sont les plus avantageux pour le vendeur (un acheteur prêt à payer plus cher offre de meilleures conditions).
- les ordres de vente les plus bas sont priorisés par rapport à ceux plus élevés car ils sont les plus avantageux pour l'acheteur (un vendeur prêt à céder moins cher offre de meilleures conditions).

Un match a lieu lorsque le prix du taker (l'ordre demandant le match) est inférieur ou égal à celui du maker pour un taker vendeur, ou supérieur ou égal à celui du maker pour un taker acheteur. La transaction a lieu au prix demandé par le maker. Pour deux *matches* équivalents en termes de prix, la priorité est donnée selon une condition temporelle (premier arrivé, premier servi). Un *matching engine* dans l'industrie est responsable de plusieurs tâches : vérifier que les prix d'échange ne sont pas complètement décorrélés du marché, s'assurer d'appliquer l'appétence au risque de l'entreprise par rapport aux quantités et au prix global des actifs achetés, etc. Pour ce mémoire, nous nous contenterons d'appliquer le *price-time priority* sur des données ingérées depuis un générateur artificiel et nous exclurons toutes les fonctionnalités annexes d'un *matching engine*.

2.1 Conception générale du *matching engine*

2.1.1 Contexte métier et technique

Notre *matching engine* est un *Proof of Concept* qui simule le cœur d'une plateforme d'échange électronique. Les acteurs du système sont de deux natures : en amont, un **générateur d'ordres synthétiques** qui représente les participants de marché (*market makers*, traders algorithmiques) en émettant des ordres d'achat et de vente ; en aval, un **système de persistance** qui consomme les transactions exécutées, jouant le rôle d'un système de reporting réglementaire.

L'enjeu qui justifie ce mémoire est le suivant : les marchés financiers modernes exigent une réactivité que les architectures logicielles classiques, telles qu'une JVM standard, ne garantissent pas nativement. Chaque microseconde de latence représente un risque financier quantifiable, notamment en cas de mouvement de marché brutal. Le *matching engine* doit donc assurer la fiabilité et le déterminisme des transactions sans intervention humaine, et ce, en continu (le moteur s'exécute sans interruption planifiée). Un jeu de données de test sera utilisé pour garantir la consistance des résultats entre les différentes versions du

moteur. L'enjeu central de ce mémoire reste l'optimisation de la latence et du déterminisme du programme.

2.1.2 Exigences fonctionnelles du *matching engine*

Le *matching engine* permet strictement de :

- générer des ordres d'achat et de vente de *commodities* (avec prix et quantité)
- annuler des ordres émis
- actualiser le prix d'un ordre émis
- autoriser des *matches* partiels (par exemple, si un acteur veut vendre 10 unités et qu'un autre souhaite en acheter 5, le *match* a lieu sur 5 unités, et l'ordre de vente reste actif pour les 5 unités restantes)
- faire le *matching* entre les ordres selon le *price-time priority*
- persister les transactions effectuées

Tout ordre qui ne trouve pas immédiatement de contrepartie n'est pas rejoué : il reste dans le carnet d'ordres (à l'état *resting*) en attendant un *match* futur ou une annulation. Il s'agit du comportement nominal d'un carnet d'ordres.

Le *matching engine* ne gère pas :

- les types d'ordres autres que le *limit order* (*market order*, *stop order*, *iceberg order* sont exclus)
- le multi-instruments : le système opère sur un instrument unique (mono-instrument). L'extension à plusieurs instruments impliquerait un carnet d'ordres et potentiellement un thread de *matching* dédiés par instrument.
- la fermeture des marchés (il tourne en continu)
- le contrôle de cohérence entre le prix d'un ordre et le marché, ni la gestion du risque (appétence au risque, exposition globale)
- la sécurité des transactions et l'authentification des acteurs
- la tolérance aux pannes et la réplication

2.1.3 Exigences non-fonctionnelles du *matching engine*

Les exigences suivantes constituent la **cible visée pour la version finale optimisée** du moteur. La *baseline* (Jalon 0), dépourvue de toute optimisation, est attendue de dépasser significativement ces seuils — notamment à cause des pauses *Stop-The-World* du *Garbage Collector* — et c'est précisément cet écart, mesuré jalon après jalon, qui constitue le fil conducteur de l'implémentation.

- **Déterminisme** : un même jeu de données d'entrée doit produire exactement les mêmes *matches*, dans le même ordre.
- **Latence** : latence P99.995 de bout en bout de l'ordre de la cinquantaine de microsecondes, et une latence maximale inférieure à la milliseconde, pour la version finale.

- **Débit** : la baseline est testée à une charge de référence de 10 000 opérations par seconde. La version finale optimisée vise un débit cible de l'ordre du million d'opérations par seconde — un objectif ambitieux mais ancré dans la littérature industrielle (LMAX revendique 6 millions d'ordres par seconde sur un seul thread en régime pleinement optimisé).
- **Bornage des prix** : les prix sont bornés à un intervalle fixe, par exemple $[0, 100\,000]$ exprimés en centimes (soit 0 à 1 000,00 unités monétaires). Ce bornage est une hypothèse de conception qui justifiera, dans les jalons avancés, l'encodage des prix en entiers (arithmétique à virgule fixe) plutôt qu'en **double**.
- **Durabilité** : pour la baseline, chaque transaction exécutée est journalisée de manière synchrone dans un fichier CSV, sur le chemin critique. Cette approche naïve, génératrice de latence, sera remise en cause au Jalon 6 par une persistance *Zero-Copy*.

2.1.4 Modèle de données

Les données fondamentales du *matching engine*, sans optimisation, sont les suivantes :

- **Side** (énumération)
 - BUY, SELL
- **OrderCommand** — représente une intention émise par le générateur d'ordres
 - **commandType** : enum (NEW, CANCEL, MODIFY)
 - **orderId** : Long
 - **side** : Side
 - **price** : Double (*borné, cf. exigences non-fonctionnelles*)
 - **quantity** : Integer (*quantité initiale pour NEW, nouvelle quantité pour MODIFY, ignorée pour CANCEL*)
 - **createdAt** : Long
- **Order** — état d'un ordre tel que conservé dans le carnet
 - **orderId** : Long
 - **side** : Side
 - **price** : Double
 - **originalQuantity** : Integer
 - **remainingQuantity** : Integer
 - **createdAt** : Long
 - **status** : enum (RESTING, PARTIALLY_FILLED, FILLED, CANCELLED)
- **Trade** — résultat d'un *match*
 - **tradeId** : Long
 - **buyOrderId** : Long
 - **sellOrderId** : Long
 - **executionPrice** : Double (*prix de l'ordre résident, cf. règle de price-time priority*)
 - **quantity** : Int

— `executedAt` : Long

Source : Stock Exchange System Design / Order Matching, Trading Engine & Scalability Explained - Youtube

2.1.5 Architecture du matching engine

Notre objectif étant d’optimiser la latence de notre programme, nous minimisons les communications réseau et inter-processus. Nous optons pour une architecture monolithique modulaire à trois modules, communiquant via une **ArrayBlockingQueue** — une structure bloquante à capacité fixe du JDK, choisie délibérément pour la baseline afin de fournir un point de comparaison rigoureux avec le *Ring Buffer* introduit au Jalon 1 :

- **Module de génération** : produit des **OrderCommand** (NEW, CANCEL, MODIFY) selon un processus de Poisson et les dépose dans la file partagée à destination du module de *matching*.
- **Module de *matching*** : consomme les **OrderCommand**, maintient le carnet d’ordres et applique le *price-time priority*. Le carnet est représenté par deux **TreeMap<Double, Deque<Order>** (un par côté), structure standard du JDK offrant une recherche en $O(\log n)$ mais dont le coût en allocations et en *pointer chasing* constituera la référence à améliorer dans les jalons suivants.
- **Module de sortie et de persistance** : reçoit les **Trade** produits par le module de *matching*, les affiche sur la sortie standard et les journalise de manière synchrone dans un fichier CSV.

Choix d’ArrayBlockingQueue plutôt que LinkedBlockingQueue. Ce choix a été arbitré entre deux structures bloquantes du JDK aux propriétés distinctes. **LinkedBlockingQueue** offre une capacité illimitée par défaut et repose sur des nœuds chaînés alloués individuellement à chaque insertion, avec deux verrous séparés pour la production et la consommation. **ArrayBlockingQueue** impose à l’inverse une capacité fixe déterminée à la construction, s’appuie sur un tableau circulaire pré-alloué, et n’expose qu’un seul verrou partagé entre les deux extrémités.

Cette dernière propriété, en apparence désavantageuse (producteur et consommateur se disputent réellement la même ressource de synchronisation), est en réalité ce qui rend **ArrayBlockingQueue** structurellement plus proche du *Ring Buffer* du Jalon 1 : capacité bornée, tableau pré-alloué, absence d’allocation de nœud par élément inséré. En retenant **ArrayBlockingQueue** pour la baseline, la comparaison de performance entre le Jalon 0 et le Jalon 1 isole plus proprement la variable réellement mesurée — le passage d’un verrou classique à un mécanisme *lock-free* — sans la mélanger à un changement simultané de stratégie d’allocation mémoire, qui n’est pas l’objet de cette comparaison.

Un second avantage, orthogonal à la comparaison inter-jalons, tient au comportement de *back-pressure*. La capacité illimitée de `LinkedBlockingQueue` masquerait un déséquilibre de débit entre producteur et consommateur : si le module de *matching* traite les commandes plus lentement qu’elles ne sont générées, la file grossirait indéfiniment sans jamais ralentir le générateur, dissimulant ainsi un comportement que la baseline devrait au contraire révéler. `ArrayBlockingQueue`, par sa capacité bornée, impose un *back-pressure* explicite : l’appel bloquant `put()` suspend le producteur dès que la file est pleine, rendant observable la saturation du système sous charge — un comportement directement comparable à celui du *Ring Buffer*, qui applique le même principe via son propre mécanisme de séquences.

La capacité retenue pour la baseline est de 1024 éléments, une puissance de deux choisie par cohérence avec le masquage binaire employé pour l’indexation circulaire du *Ring Buffer* au Jalon 1.

Source : How Does a Matching Engine Work - Juan D Bustamante - Devnexus 2022 - Youtube

2.2 *Continuous Benchmarking* pour le *matching engine*

2.2.1 Contexte général

Pour le développement de notre *matching engine*, nous avons adopté le principe du *continuous benchmarking*, qui se distingue de l’intégration continue classique en intégrant une phase de validation des performances à chaque jalon, garantissant qu’aucune modification du code ne dégrade silencieusement les acquis des itérations précédentes. Cette exigence empêche, dans un contexte de trading haute fréquence, qu’une régression de quelques microsecondes passe inaperçue. Cela représenterait un risque financier quantifiable, et non un simple inconfort de performance. La référence sur laquelle s’appuient les comparaisons est définie manuellement, afin d’éviter qu’un commit non pertinent, un refactoring mineur ou une correction de documentation, ne vienne silencieusement déplacer cette référence et propager du bruit de mesure vers les comparaisons futures.

Concernant la métrique retenue, nous ciblons la latence P99.99. L’idéal aurait été d’utiliser le percentile P99.995, cohérent avec les exigences non-fonctionnelles définies en section 2.1.3 ; ce percentile n’étant pas disponible nativement au sein de JMH, nous l’intégrerons ultérieurement via les histogrammes HDR.

Enfin, la politique de régression repose sur deux seuils : au-delà de 20%, le code est rejeté, signalant une rupture structurelle incompatible avec nos objectifs ; entre 0% et 20%, un avertissement est émis. Ce palier assure une flexibilité opérationnelle en signalant les dérives sans bloquer les contributions dont le bénéfice global reste positif.

2.2.2 Architecture du pipeline de *Continuous Benchmarking*

Le pipeline a pour objectif de garantir qu’aucune régression de performance ne franchit la barrière de la revue de code. Il s’exécute sur GitHub Actions à partir de trois déclencheurs : un *push* sur *main*, une *pull request*, ou un déclenchement manuel (*workflow_dispatch*). Ce dernier mode est essentiel : il permet de rejouer le pipeline sur un commit précis, indépendamment de l’historique, afin de valider ou d’invalidier manuellement une baseline candidate.

À l’échelle macro, le pipeline est structuré en trois jobs séquentiels à responsabilités disjointes, comme l’illustre la figure 6. À l’échelle micro, chaque job délègue à un outil spécialisé :

- **Job 1** — *Correctness gate* (*build-and-test*) : Maven compile le projet et exécute la suite de tests unitaires via `mvn -B verify`. Les jobs de performance ne s’exécutent que si ce verrou de correction fonctionnelle est franchi.
- **Job 2** — *Performance gate* (*benchmark*) : les tests unitaires sont délibérément sautés (`-DskipTests`) pour ne pas dupliquer le travail du job précédent. JMH exécute les benchmarks et produit un fichier `jmh-result.json` au format JSON natif, publié en artefact de pipeline afin de découpler la production du résultat de son analyse.
- **Job 3** — *Regression gate* (*compare-baseline*) : le script `compare-benchmarks.sh` télécharge l’artefact et le compare à la baseline versionnée dans le dépôt (`benchmarks/baseline.json`). Le résultat est publié dans le **Job Summary** de GitHub Actions, directement visible dans l’interface du run sans permissions supplémentaires.

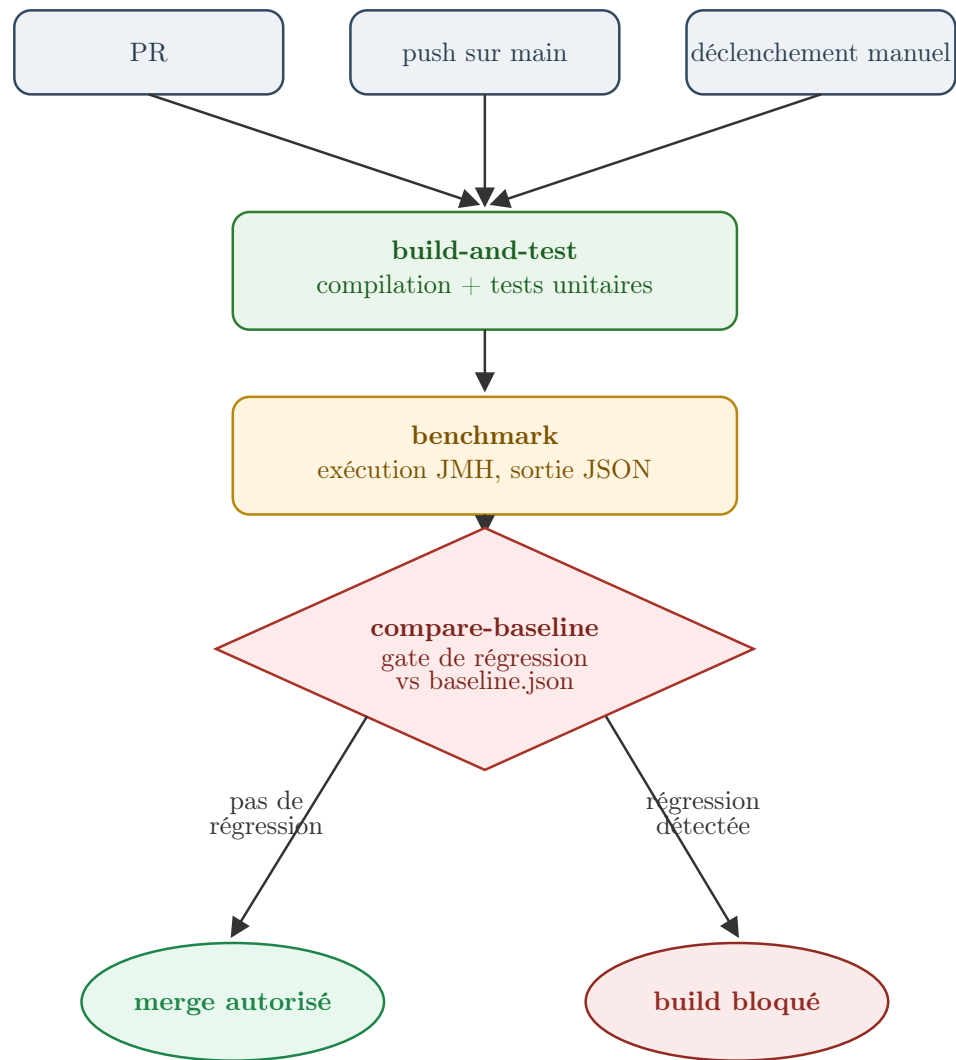


FIGURE 6 – Pipeline *Continuous Benchmarking* du *matching engine*

2.3 Jalon 0 - L'Order Matching Engine en Programmation Orientée Objets classique et ses inconvénients

2.3.1 Analyse de l'OrderBook et de sa structure avec Java Object Layout

L'outil *Java Object Layout* (JOL) permet d'inspecter la disposition mémoire réelle d'un objet Java, telle que la JVM la matérialise, plutôt que de la déduire de la seule déclaration des champs. La figure 7 présente cette disposition pour `Order`, l'unité de base répétée à chaque niveau de prix du carnet.

```
com.victorien.matchingengine.model.Order object internals:
OFFSET  SIZE        TYPE DESCRIPTION           VALUE
  0      4         (object header)           01 00 00 00 (00000001 00000000 00000000 00000000) (1)
  4      4         (object header)           00 00 00 00 (00000000 00000000 00000000 00000000) (0)
  8      4         (object header)           20 35 00 01 (00100000 00110101 00000000 00000001) (16790816)
 12      4             int Order.originalQuantity      10
 16      8             long Order.orderId        1
 24      8             double Order.price         100.0
 32      8             long Order.createdAt      1676419200
 40      4             int Order.remainingQuantity 10
 44      4 com.victorien.matchingengine.model.Side Order.side (object)
 48      4 com.victorien.matchingengine.model.Order.Status Order.status (object)
 52      4             (loss due to the next object alignment)

Instance size: 56 bytes
Space losses: 0 bytes internal + 4 bytes external = 4 bytes total
```

FIGURE 7 – Disposition mémoire de `Order` obtenue avec Java Object Layout

Trois éléments ressortent de cette analyse :

- L'en-tête d'objet (*object header*) occupe 12 des 56 octets de l'instance. Aussi, l'alignement mémoire de la JVM introduit une perte supplémentaire de 4 octets en fin d'instance, du fait de l'*object alignment* suivant. Ensemble, ces deux postes représentent 16 octets sur 56, soit 29% de la taille de l'instance : un coût fixe payé par chaque `Order`, indépendant du nombre de champs métier, et qui pèsera d'autant plus lourd que le carnet contiendra un grand nombre d'ordres simultanément résidents.
- Un autre élément, moins visible mais tout aussi structurant : l'ordre des champs dans le *layout* ne correspond pas à l'ordre de déclaration dans la classe. `originalQuantity` (offset 12, `int`) précède `orderId` (offset 16, `long`) et `price` (offset 24, `double`), alors que `orderId` est déclaré avant `originalQuantity` dans le code source. La JVM HotSpot réordonne les champs pour minimiser le *padding* interne, en regroupant les champs de 8 octets (`long`, `double`), puis ceux de 4 octets (`int`), puis les références (4 octets en *compressed oops*) — un comportement documenté, mais qui échappe entièrement au contrôle du développeur. Conséquence directe : le *layout* mémoire d'un objet Java standard n'est pas pilotable depuis le code source, ce qui rend toute optimisation de densité mémoire impossible sans sortir du modèle objet, par exemple via des tableaux de primitifs.
- Enfin, les champs `side` (offset 44) et `status` (offset 48) n'occupent chacun que 4 octets, mais ces octets ne contiennent pas la valeur de l'énumération — ils contiennent une référence vers un objet `enum` alloué ailleurs dans le

tas. Lire le `Side` d'un `Order` suppose donc de charger l'objet `Order`, puis de suivre cette référence jusqu'à l'objet `Side` correspondant : une forme de *pointer chasing* en miniature, potentiellement coûteuse en défauts de cache lorsqu'elle est répétée à chaque ordre du carnet lors d'un parcours.

Ces observations forment, ensemble, la justification empirique de la transition vers le *Data-Oriented Design* envisagée aux jalons suivants : 56 octets par `Order`, dont 16 octets d'*overhead* non métier (29%) ; un *layout* mémoire non contrôlable, réordonné par la JVM selon ses propres règles d'alignement ; et une indirection supplémentaire introduite par chaque champ `enum`. Un modèle *Structure-of-Arrays*, où `side` deviendrait par exemple un simple `byte` stocké dans un tableau contigu, ferait disparaître à la fois l'*overhead* d'en-tête par élément et le *pointer chasing* des énumérations.

La baseline du Jalon 0 repose sur une structure volontairement non optimisée du JDK : un carnet d'ordres par côté, indexé à la fois par prix et par identifiant. Ce choix de structure duale illustre un compromis classique entre deux besoins d'accès orthogonaux, et sert de point de comparaison pour les optimisations des jalons suivants.

Structure duale : niveaux de prix triés et index par identifiant.

Chaque côté du carnet est représenté par un `TreeMap<Double, Deque<Order>>`, qui maintient les niveaux de prix triés et, à l'intérieur de chaque niveau, une file FIFO matérialisant la priorité temporelle (*price-time priority*). Un `HashMap<Long, Order>` auxiliaire est nécessaire en complément : un `TreeMap` trié par prix ne permet pas de retrouver un ordre par son identifiant en temps logarithmique, or l'annulation d'un ordre (`CANCEL`) doit pouvoir le localiser sans parcourir tous les niveaux de prix.

```
public class OrderBook {

    private final TreeMap<Double, Deque<Order>> buyLevels =
        new TreeMap<>(Comparator.reverseOrder());
    private final TreeMap<Double, Deque<Order>> sellLevels =
        new TreeMap<>();

    private final Map<Long, Order> ordersById = new HashMap
        <>();

    public void insert(Order order) {
        if (Objects.isNull(order))
            throw new IllegalArgumentException("Order cannot
                be null");
        if (this.ordersById.containsKey(order.orderId()))
            throw new IllegalArgumentException("Order
                already exists");

        this.levelsFor(order.side())
```

```

        .computeIfAbsent(order.price(), s -> new
            ArrayDeque<>())
        .offerLast(order);
    this.ordersById.put(order.orderId(), order);
}

public Order remove(long orderId) {
    Order order = this.ordersById.get(orderId);
    if (Objects.isNull(order))
        return null;

    this.ordersById.remove(orderId);
    this.levelsFor(order.side()).get(order.price())
        .removeIf((Order toRemove) -> toRemove.
            orderId() == orderId);

    if (this.levelsFor(order.side()).get(order.price()).
        isEmpty())
        this.levelsFor(order.side()).remove(order.price
            ());

    return order;
}
}

```

Listing 5 – Structure duale de l'OrderBook : niveaux de prix et index par identifiant

Deux gardes protègent la cohérence de la structure duale dès l'insertion : un ordre `null` est rejeté immédiatement, et un `orderId` déjà présent dans l'index lève une exception plutôt que d'écraser silencieusement l'entrée existante — ce qui laisserait l'ordre original orphelin dans sa file de niveau de prix, visible par `bestBid()` ou `bestAsk()` mais inaccessible via `remove()`.

Un point d'attention sémantique mérite d'être souligné : les deux côtés n'ont pas le même ordre naturel. Le côté **BUY** doit exposer le prix le plus élevé en tête (le meilleur acheteur), le côté **SELL** le prix le plus bas en tête (le meilleur vendeur). Un `TreeMap` trie par défaut en ordre croissant : c'est pourquoi `buyLevels` est construit avec un comparateur inversé (`Comparator.reverseOrder()`), tandis que `sellLevels` conserve l'ordre naturel.

Cette asymétrie a un coût explicite sur `remove` : la localisation du niveau de prix via `ordersById` s'effectue en $O(\log n)$, mais le retrait proprement dit, via `removeIf` sur la `Deque` du niveau, s'exécute en $O(k)$, k étant le nombre d'ordres présents à ce niveau de prix, puisque l'ordre à retirer n'est pas nécessairement en tête de file. Cette dégradation linéaire sur les annulations est une limitation documentée de la baseline, mesurable sous forte contention, et constitue l'un des points que les jalons d'optimisation suivants chercheront à réduire.

Accès au meilleur prix. `bestBid()` et `bestAsk()` délèguent à une méthode privée commune, qui lit le premier niveau de prix de la structure et retourne l'ordre en tête de la file de ce niveau :

```
private Order maxOrder(Side side) {
    if (this.levelsFor(side).isEmpty())
        return null;

    Order maxOrder = this.levelsFor(side).firstEntry().
        getValue().peekFirst();

    if (Objects.isNull(maxOrder))
        throw new IllegalStateException(
            "Price level exists with no orders - internal
            state violated");

    return maxOrder;
}
```

Listing 6 – Lecture du meilleur prix et de l'ordre prioritaire au sein d'un niveau

L'ordre en tête de file (`peekFirst()`) est le plus ancien du niveau de prix sous l'hypothèse que `createdAt` croît avec l'ordre d'insertion, c'est-à-dire que les ordres sont soumis au moteur dans l'ordre chronologique de leur horodatage. Dans ce cas, `insert` qui ne pousse qu'en fin de file (`offerLast`) garantit la priorité temporelle par construction, sans qu'il soit nécessaire de parcourir le niveau à la lecture. La garde `IllegalStateException` défend un invariant interne : un niveau de prix présent dans le `TreeMap` ne devrait jamais être vide, puisque `remove` le retire dès qu'il se vide.

2.3.2 L'état d'un ordre : la classe `Order`

Contrairement à `OrderCommand` et `Trade`, immuables, `Order` porte un état qui évolue dans le temps : `remainingQuantity` diminue au fil des exécutions partielles, et `status` transite de `RESTING` vers `PARTIALLY_FILLED` puis `FILLED`, ou vers `CANCELLED` sur annulation. Deux invariants métier sont protégés explicitement par exception plutôt que par assertion ou ignorance silencieuse, un choix de conception justifié par le contexte : un ordre déjà exécuté ou annulé qui serait modifié à nouveau silencieusement introduirait une incohérence comptable difficile à détecter a posteriori.

```
public void fill(int filledQuantity) {
    if (this.status == Status.CANCELLED)
        throw new IllegalStateException("The order has been
            cancelled");
    if (this.status == Status.FILLED)
        throw new IllegalStateException("The order has been
            filled");
    if (filledQuantity <= 0)
```

```

        throw new IllegalArgumentException(
            "Cannot ask for a negative or a zero quantity of
            an asset");
    if (filledQuantity > this.remainingQuantity)
        throw new IllegalArgumentException(
            "Cannot ask for more than remaining quantity of
            an asset");

    this.remainingQuantity -= filledQuantity;
    this.status = this.remainingQuantity == 0 ? Status.
        FILLED : Status.PARTIALLY_FILLED;
}

public void cancel() {
    if (this.status == Status.CANCELLED)
        throw new IllegalStateException("Order already
            cancelled");
    if (this.status == Status.FILLED)
        throw new IllegalStateException("Order already
            filled");

    this.status = Status.CANCELLED;
}

```

Listing 7 – Transitions d'état de l'ordre : exécution et annulation

2.3.3 Le moteur de matching : la classe MatchingEngine

Architecture générale. MatchingEngine est le point d'entrée unique du moteur. Sa responsabilité est de traduire des OrderCommand en Trade en appliquant la *price-time priority*, en déléguant entièrement la gestion de la structure du carnet à OrderBook. La méthode publique process() dispatche chaque commande vers le handler approprié selon son type :

```

public class MatchingEngine {

    private final OrderBook orderBook = new OrderBook();
    private final AtomicLong tradeIdGenerator = new
        AtomicLong(0);

    public List<Trade> process(OrderCommand command) {
        List<Trade> trade = new ArrayList<>();
        switch (command.commandType()) {
            case NEW -> trade = handleNew(command);
            case CANCEL -> trade = handleCancel(command);
            case MODIFY -> trade = handleModify(command);
        }
        return trade;
    }
}

```

```
}
```

Listing 8 – Point d’entrée unique du moteur de matching

Le générateur d’identifiants de trade est un `AtomicLong`, choix qui anticipe une éventuelle montée en concurrence au Jalon 1 sans introduire de synchronisation coûteuse dans la baseline mono-thread.

Traitement d’une commande NEW. L’ordre entrant est d’abord inséré dans le carnet — il devient résident — puis la boucle de matching est déclenchée immédiatement. Cette approche garantit que si l’ordre n’est que partiellement exécuté, sa fraction résiduelle reste dans le carnet en attente d’une contrepartie future.

```
private List<Trade> handleNew(OrderCommand command) {
    Order incoming = new Order(
        command.orderId(), command.side(),
        command.price(), command.quantity(), command.
            createdAt());
    this.orderBook.insert(incoming);
    return this.match(incoming);
}
```

Listing 9 – Traitement d’une commande NEW

La boucle de matching. La méthode `match()` reçoit le taker en paramètre — c’est toujours l’ordre entrant, qui ne change pas au fil de la boucle. À chaque itération, le meilleur ordre du côté opposé est récupéré dynamiquement : si le maker précédent a été épuisé et retiré du carnet, `bestCounterparty()` renvoie automatiquement le suivant. La boucle s’arrête dès que le taker est épuisé, que le carnet est vide du côté opposé, ou que les prix ne se croisent plus.

```
private List<Trade> match(Order taker) {
    List<Trade> trades = new ArrayList<>();

    while (taker.remainingQuantity() > 0) {
        Order maker = bestCounterparty(taker);

        if (maker == null || !isCrossed(taker, maker))
            break;

        int quantity = Math.min(taker.remainingQuantity(),
            maker.remainingQuantity());

        taker.fill(quantity);
        maker.fill(quantity);

        trades.add(buildTrade(taker, maker, quantity));
    }
}
```

```

        if (maker.isEmpty())
            orderBook.remove(maker.orderId());

        if (taker.isEmpty())
            orderBook.remove(taker.orderId());
    }

    return trades;
}

```

Listing 10 – Boucle de matching : price-time priority

Le fait de récupérer `bestCounterparty(taker)` en tête de chaque itération plutôt qu’une seule fois avant la boucle suffit à gérer la progression : si le maker a été retiré du carnet à la fin de l’itération précédente, l’appel suivant retourne naturellement le prochain meilleur ordre disponible. Le taker, lui, reste inchangé tant qu’il a de la quantité résiduelle.

Méthodes auxiliaires. La boucle délègue trois responsabilités à des méthodes privées distinctes :

```

private boolean isCrossed(Order taker, Order maker) {
    return taker.side() == Side.BUY
        ? taker.price() >= maker.price()
        : taker.price() <= maker.price();
}

private Trade buildTrade(Order taker, Order maker, int
    quantity) {
    long id = tradeIdGenerator.getAndIncrement();
    long executionTime = Instant.now().getEpochSecond();
    if (taker.side() == Side.BUY)
        return new Trade(id, taker.orderId(), maker.orderId()
            (),
                maker.price(), quantity,
                executionTime);
    else
        return new Trade(id, maker.orderId(), taker.orderId()
            (),
                maker.price(), quantity,
                executionTime);
}

private Order bestCounterparty(Order order) {
    return order.side() == Side.BUY
        ? this.orderBook.bestAsk()
        : this.orderBook.bestBid();
}

```

Listing 11 – Méthodes auxiliaires du matching

`isCrossed()` se limite à la comparaison de prix : la vérification de quantité résiduelle est portée par la condition de boucle (`taker.remainingQuantity() > 0`) et par le `null`-check sur le maker, ce qui évite toute redondance. `buildTrade()` isole la construction du `Trade` et l'attribution correcte des rôles `buyOrderId` et `sellOrderId` selon le `side` du taker ; le prix d'exécution est toujours celui du maker, conformément à la règle de *price-time priority*. À noter : l'appel à `Instant.now()` sur le chemin critique est une limitation documentée de la *baseline* — il effectue un appel système à chaque trade produit, source de latence non déterministe, et sera remplacé au Jalon 1 par un timestamp injecté directement dans `OrderCommand` côté générateur. `bestCounterparty()` exprime en une ligne l'accès au meilleur ordre du côté opposé.

Traitement des commandes CANCEL et MODIFY.

```
private List<Trade> handleCancel(OrderCommand command) {
    Order removed = this.orderBook.remove(command.orderId());
    ;
    if (removed == null)
        throw new IllegalArgumentException(
            "Cannot cancel unknown order: " + command.
                orderId());
    removed.cancel();
    return new ArrayList<>();
}

private List<Trade> handleModify(OrderCommand command) {
    Order existing = this.orderBook.remove(command.orderId());
    ;
    if (existing == null)
        throw new IllegalArgumentException(
            "Cannot modify unknown order: " + command.
                orderId());
    existing.cancel();
    Order modified = new Order(
        command.orderId(), existing.side(),
        command.price(), existing.remainingQuantity(),
        command.createdAt());
    this.orderBook.insert(modified);
    return this.match(modified);
}
```

Listing 12 – Handlers CANCEL et MODIFY

`handleCancel()` retire l'ordre du carnet puis appelle `cancel()` sur l'objet, ce qui met à jour son statut vers `CANCELLED` et protège contre toute utilisation ultérieure de l'instance. Tenter d'annuler un ordre inconnu lève une `IllegalArgumentException` plutôt que de retourner silencieusement une liste vide, afin d'exposer immédiatement les incohérences d'état entre le producteur de commandes et le moteur.

`handleModify()` applique un mécanisme *remove-then-reinsert* : l'ordre existant est retiré et annulé, puis un nouvel ordre est créé avec le nouveau prix, en conservant la `remainingQuantity` résiduelle plutôt que la quantité originale. Ce choix a une conséquence sémantique forte : l'ordre modifié perd sa priorité temporelle, puisqu'il est réinséré en fin de file à son niveau de prix avec le `createdAt` de la commande `MODIFY`. Modifier un prix équivaut donc à annuler l'ordre et en soumettre un nouveau, ce qui peut déclencher un match immédiat si le nouveau prix croise le carnet.

2.3.4 Le pipeline producteur-consommateur

La baseline assemble trois modules sur trois threads distincts, reliés par deux `BlockingQueue` bornées. Ce modèle producteur-consommateur isole chaque responsabilité sur son propre thread tout en garantissant la propagation d'un signal d'arrêt ordonné (*poison pill pattern*).

Architecture générale. Le pipeline est le suivant :

1. `OrderGenerator` génère des `OrderCommand` et les dépose dans `commandQueue`.
2. `MatchingWorker` consomme les commandes, les traite via `MatchingEngine`, et dépose les `Trade` produits dans `tradeQueue`.
3. `PersistenceWorker` consomme les trades et les journalise dans un fichier CSV.

Chaque étage se termine proprement dès réception d'une *poison pill* — une valeur sentinelle à `orderId = -1` pour les commandes, à `tradeId = -1` pour les trades — qu'il propage vers l'étage suivant avant de s'arrêter, garantissant qu'aucun thread aval ne reste bloqué indéfiniment sur un `take()`.

Génération des commandes : `OrderGenerator`. `OrderGenerator` modélise un flux d'arrivées selon un processus de Poisson de taux λ (`targetRatePerSecond`), en tirant les inter-arrivées selon une loi exponentielle par inversion de la fonction de répartition :

```
private long nextInterArrivalMillis() {
    double u = random.nextDouble();
    double interArrivalSeconds = -Math.log(1 - u) /
        targetRatePerSecond;
    return (long) (interArrivalSeconds * 1000);
}

@Override
public void run() {
    try {
        for (int i = 0; i < commandCount; i++) {
            outputQueue.put(randomCommand());
        }
    }
}
```

```

        Thread.sleep(nextInterArrivalMillis());
    }
    outputQueue.put(SHUTDOWN_SIGNAL);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    Logger.getLogger(OrderGenerator.class.getName())
        .log(Level.SEVERE, "Thread interrupted
            unexpectedly");
}
}

```

Listing 13 – Tirage de l’inter-arrivée et boucle de génération

Une limitation documentée mérite d’être soulignée : `Thread.sleep()` ne résout que des délais entiers en millisecondes. Pour un taux cible de 10 000 ordres/seconde, l’inter-arrivée moyenne théorique est de 0,1 ms — valeur tronquée à 0 ms par la conversion en `long` dans 99,995 % des cas. À ce taux, le générateur produit donc en boucle quasi continue plutôt que d’approximer un vrai processus de Poisson. Ce comportement est accepté comme limitation documentée de la baseline : une implémentation précise à haute fréquence (via `LockSupport.parkNanos()`) consommerait un cœur CPU en continu pour le seul rythme des arrivées, ce qui fausserait les mesures de charge du système sous test. Le *seed* du générateur aléatoire est fixée à 42, ce qui garantit le déterminisme du jeu de commandes entre deux exécutions — condition nécessaire pour que les tests de régression soient reproductibles.

Routage et propagation du signal d’arrêt : `MatchingWorker`.

```

@Override
public void run() {
    boolean shutdownPropagated = false;
    try {
        while (true) {
            OrderCommand inputCommand = inputQueue.take();
            if (inputCommand.equals(OrderGenerator.
                SHUTDOWN_SIGNAL)) {
                outputQueue.put(MatchingWorker.
                    SHUTDOWN_SIGNAL);
                shutdownPropagated = true;
                break;
            }
            for (Trade trade : engine.process(inputCommand))
                outputQueue.put(trade);
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        Logger.getLogger(MatchingWorker.class.getName())
            .log(Level.SEVERE, "Thread interrupted
                unexpectedly");
    }
}

```

```

    } finally {
        if (!shutdownPropagated)
            propagateShutdownBestEffort();
    }
}

```

Listing 14 – Boucle principale de MatchingWorker

Le flag `shutdownPropagated` garantit qu'un seul signal d'arrêt est déposé dans `outputQueue`, quel que soit le chemin de sortie. Sur le chemin nominal, la *poison pill* est propagée avant le `break`. Sur le chemin d'interruption, le `finally` délègue à `propagateShutdownBestEffort()`, qui utilise `offer()` plutôt que `put()` : `put()` vérifierait le flag d'interruption déjà positionné et lèverait immédiatement une `InterruptedException`, laissant `PersistenceWorker` bloqué indéfiniment.

Persistance synchrone : PersistenceWorker.

```

@Override
public void run() {
    try {
        csvWriter.write(CSV_HEADER);
        Trade trade;
        while (true) {
            trade = inputQueue.take();
            if (trade.equals(MatchingWorker.SHUTDOWN_SIGNAL)
            )
                break;
            System.out.println(trade);
            csvWriter.write(toCsvLine(trade));
            csvWriter.flush();
        }
    } catch (IOException e) {
        Logger.getLogger(PersistenceWorker.class.getName())
            .log(Level.SEVERE, "Unexpected I/O error
                occurred while writing to file");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        Logger.getLogger(PersistenceWorker.class.getName())
            .log(Level.SEVERE, "Thread interrupted
                unexpectedly");
    }
}

private String toCsvLine(Trade trade) {
    return String.format(Locale.US, "%d,%d,%d,%.2f,%d,%d\n",
        trade.tradeId(), trade.buyOrderId(), trade.
            sellOrderId(),
            trade.executionPrice(), trade.quantity(), trade.

```

```

        executedAt());
    }

```

Listing 15 – Boucle de persistance CSV

Chaque trade est écrit puis immédiatement "flushé" (`flush()` après chaque ligne), conformément à l'exigence de durabilité des NFR : en cas d'arrêt brutal du processus, les trades déjà traités ne sont pas perdus dans un buffer. `Locale.US` est passé explicitement à `String.format()` pour forcer le point comme séparateur décimal indépendamment de la locale système — sans quoi un environnement francophone produirait 108,63 au lieu de 108.63, créant une colonne fantôme dans le CSV. La fermeture du `Writer` est déléguée à l'appelant via `try-with-resources` dans `Main`.

Assemblage du pipeline : `Main`.

```

public static void main(String[] args) throws
    InterruptedException, IOException {
    BlockingQueue<OrderCommand> commandQueue = new
        ArrayBlockingQueue<>(QUEUE_CAPACITY);
    BlockingQueue<Trade> tradeQueue = new ArrayBlockingQueue
        <>(QUEUE_CAPACITY);

    MatchingEngine engine = new MatchingEngine();
    String outputCsvPath = buildOutputCsvPath();

    try (Writer csvWriter = new FileWriter(outputCsvPath)) {
        Thread generatorThread = new Thread(
            new OrderGenerator(commandQueue,
                COMMAND_COUNT, TARGET_RATE_PER_SECOND),
            "order-generator");
        Thread matchingThread = new Thread(
            new MatchingWorker(commandQueue, tradeQueue,
                engine),
            "matching-worker");
        Thread persistenceThread = new Thread(
            new PersistenceWorker(tradeQueue, csvWriter)
            ,
            "persistence-worker");

        generatorThread.start();
        matchingThread.start();
        persistenceThread.start();

        generatorThread.join();
        matchingThread.join();
        persistenceThread.join();
    }
}

```

```

        System.out.println("Pipeline terminé. Trades journalisés
                           dans " + outputCsvPath);
    }

    private static String buildOutputCsvPath() throws
        IOException {
        Files.createDirectories(OUTPUT_DIR);
        String timestamp = LocalDateTime.now().format(
            TIMESTAMP_FORMAT);
        return OUTPUT_DIR.resolve("trades-" + timestamp + ".csv"
            ).toString();
    }
}

```

Listing 16 – Assemblage et lancement du pipeline

Les deux queues sont des `ArrayBlockingQueue` bornées à `QUEUE_CAPACITY = 1024`, ce qui exerce une contre-pression naturelle sur le générateur : si le matching ne suit pas, `put()` bloque le producteur plutôt que d'accumuler des commandes en mémoire sans limite. Le fichier de sortie est nommé par horodatage (`trades-yyyyMMdd-HHmss-SSS.csv`), ce qui évite l'écrasement entre deux exécutions successives sans nécessiter de compteur d'état global. La fermeture du `Writer` est gérée par un `try-with-resources`, garantissant qu'elle intervient après la terminaison de `persistenceThread` — aucune écriture résiduelle ne peut donc survenir sur un `writer` déjà fermé.

2.3.5 Mesure de la baseline : méthodologie JMH et interprétation des résultats

Choix de l'outillage. La mesure de la latence de la fonction principale, `engine.process`, repose sur JMH. Cet outil permet d'éviter les pièges classiques du micro-benchmarking en Java (*dead-code elimination*, biais de warmup, etc.). Utiliser `System.nanoTime()` produirait des mesures optimistes sur plusieurs ordres de grandeur. Le JIT verrait que le résultat de `engine.process` n'est jamais utilisé et supprimerait simplement l'appel.

Configuration commune des trois benchmarks. Les annotations de classe de `MatchingEngineBenchmark` fixent le protocole appliqué aux trois scénarios :

```

@BenchmarkMode(Mode.SampleTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
@Warmup(iterations = 5, time = 1, timeUnit = TimeUnit.
    SECONDS)
@Measurement(iterations = 10, time = 1, timeUnit = TimeUnit.
    SECONDS)
@Fork(1)
public class MatchingEngineBenchmark { ... }

```

Listing 17 – Configuration JMH du benchmark de la baseline

`SampleTime` est privilégié à `AverageTime` car il permet de voir les percentiles P50, P90, P99, P99.9, P99.99, P99.999, P99.9999 et P100 permettent d'éviter les problèmes de latence masqués par la moyenne comme vu dans l'état de l'art. Cinq itérations de *warmup* d'une seconde laissent au compilateur le temps aux méthodes du *hot path* (`process()`, `match()`, `bestCounterparty()`) d'être optimisées par le JIT avant les mesures. Dix itérations de mesure d'une seconde constituent les mesures de notre benchmark.

L'annotation `@Fork()` permet de lancer plusieurs JVM pour mitiger les biais de la JVM (variation des décisions JIT,) en faisant tourner successivement les mesures dans des JVMs distinctes et en moyennant les résultats pour limiter les impacts des aléas de la JVM sur les mesures. Le choix de `@Fork(1)` permet d'isoler les biais de la JVM (variation des décisions JIT, état partagé du garbage collector) mais ne les amortit pas. Cela permet cela-dit d'obtenir une vitesse d'exécution des mesures. Il est envisagé de lancer plusieurs JVM pour les mesures comparatives finales (par exemple lancer cinq JVM en parallèle).

Trois scénarios pour trois profils de latence.

Nous avons choisi de distinguer trois scénarios distincts :

- `restingOrder` : le carnet est vide et l'ordre entrant ne trouve jamais de correspondance mais est simplement enregistré. On mesure, ici, le coût d'une insertion suivi d'une tentative de match avortée.
- `fullMatch` : un *maker* de quantité identique est préparé dans le `@Setup`, le *taker* est mis en correspondance avec une contrepartie exactement une fois. À la fin d'une itération, un `Trade` est produit et les deux `Order` sont retirés du carnet.
- `partialMatch` : la quantité du taker excède celle du maker. Le *maker* est totalement consommé et retiré tandis que le *taker* est partiellement rempli.

Fusionner ces trois scénarios en un seul masquerait des profils de latence différents derrière une moyenne simplificatrice. C'est précisément cette simplification trompeuse que nous souhaitons éviter. Trois `@Benchmark` distincts couvrent donc les trois chemins possibles à l'intérieur de `match()`.

Un `@Setup(Level.Invocation)` nécessaire.

Le choix du niveau de *setup* est important, ici nous avons choisi `Level.Invocation` qui réinitialise l'*engine* avant chaque mesure pour garantir que les `TreeMap` et la `HashMap` de l'`OrderBook` conservent une taille stable d'un appel à l'autre. Sans cette précaution, ces structures de données grossiraient au fil des nombreux appels JMH et augmenteraient mécaniquement la latence des opérations, faussant nos résultats.

Neutralisation du *dead code elimination*.

Chaque benchmark publie le résultat de `process()` dans un `Blackhole` passé en paramètre :

```
blackhole.consume(state.engine.process(incoming));
```

Listing 18 – Neutralisation du DCE via Blackhole

Sans cette consommation explicite, le JIT penserait que la valeur de retour n’est jamais lue et pourrait supprimer toutes les opérations. `Blackhole.consume()` force le compilateur à considérer le résultat comme lu.

Résultats bruts (JMH 1.37, OpenJDK 26.0.1, macOS ARM64). Les distributions mesurées sur la baseline sont résumées dans la table 4 extraite du champ `scorePercentiles` de `jmh-result.json`.

Scénario	Moyenne	P50	P90	P99	P99.9	P99.99	P99.999	P100
<code>restingOrder</code>	44.6	42	42	84	416	6 138	146 654	324 096
<code>partialMatch</code>	80.0	83	84	167	917	9 642	443 855	471 552
<code>fullMatch</code>	91.3	83	125	208	709	10 076	134 567	540 672

TABLE 4 – Latences par percentile du chemin critique `process()`, en ns/op. Extraites de `jmh-result.json`.

Interprétation. Cinq observations se dégagent de ces chiffres.

- Premièrement, la plus faible médiane d’exécution se déroule en une quarantaine de nanosecondes. `restingOrder` présente un P50 à 42 ns soit 168 cycles d’horloges (selon l’hypothèse de l’état de l’art d’un cycle d’horloge valant 0.25 ns). Une opération vaut donc moins qu’un accès à la RAM en terme de coût. Il s’agit d’une performance correcte dès la baseline lié à la petite taille de nos dictionnaires. Le chemin d’exécution a donc une complexité raisonnable. Le coût sur lequel nous nous focaliserons se situera donc en queue de distribution.
- Deuxièmement, la queue de distribution s’étale sur plusieurs ordres de grandeur (on passe de 10^2 pour le P99.9 à 10^5 pour le P99.999). Ce changement d’échelle immense ne s’explique pas par une augmentation de la complexité des opérations mais par des événements externes documentées dans l’état de l’art (pauses *Stop-the-World*, changement de contexte du fait des verrous). Il dévoile un besoin d’optimisation réel des performances pour se rapprocher de nos objectifs.
- Troisièmement, à partir du P99.999 on voit que le `partialMatch` devient près de quatre fois plus lent que le `fullMatch` tandis qu’il était plus performant sur tous les précédents percentiles. La cause de cette sous-performance n’est pas tranchée et mérite une investigation dédiée.
- Quatrièmement, la gigue mesurée par JMH est significative. L’intervalle de confiance rapporté pour `fullMatch` est [84.30 ; 98.24] ns/op au-

tour d’une moyenne de 91.27 ns/op, soit une demi-largeur d’environ $\pm 7\%$. Pour `partialMatch`, l’intervalle est [70.55 ; 89.47] ns/op autour de 80.01 ns/op, soit environ $\pm 12\%$. Ces marges importantes s’expliquent par l’imprévisibilité des pauses *Stop-the-World* ainsi que celle des changements de contexte. Elles appellent à des optimisations de la gigue pour améliorer le déterminisme du programme.

- Cinquièmement, les chiffres confirment notre intuition sur la nécessité de séparer les mesures en trois scénarios pour ne pas agglomérer des réalités différentes en un seul chiffre simplificateur.

Cette baseline chiffrée, complétée par l’analyse mémoire de la figure 7, constitue le point de référence quantitatif auquel les jalons suivants viendront se comparer. Il permet avec des mesures empiriques de confirmer les affirmations de l’état de l’art et montre le besoin d’aller au-delà des architectures à verrou et du paradigme objet pour la faible latence. Ces mesures locales constituent la référence de baseline retenue pour ce mémoire ; le pipeline CI/CD, aligné sur la même version de JDK, sert uniquement de garde de non-régression sur les jalons ultérieurs, sans prétention à reproduire l’environnement de mesure exact.

Bibliography

- [1] Ulrich Drepper. What Every Programmer Should Know About Memory, 2007
- [2] Jhon L. Hennessy, David A. Patterson Computer Architecture : A Quantitative Approach, 2017
- [3] Benjamin J. Evans. Optimizing Java : Practical techniques for improving JVM application performance, 2018
- [4] Tene Gil. Understanding Java Garbage Collection, 2015
- [5] Tene Gil. How not to measure latency, 2015
- [6] Thompson Martin, et al. Disruptor : High performance alternative to bounded queues for exchanging data between concurrent threads, 2011
- [7] Trisha Gee et Mike Barker, Understanding the Disruptor, a Beginner's Guide to Hardcore Concurrency
- [8] Trisha Gee, Dissecting the Disruptor : Why it's so fast (part two) – Magic cache line padding
- [9] Brendan Gregg. Systems Performance, 2020
- [10] FIX Trading Community. Simple Binary Encoding Specification, 2019
- [11] Documentation Chronicle-core
- [12] Documentation technique d'Aeron Messaging
- [13] Shipilëv Aleksey. Java Object Layout (JOL)
- [14] Shipilëv Aleksey. Java Objects Inside Out
- [15] Documentation. OpenJDK
- [16] Aeron : Efficient Reliable UDP Unicast and Multicast Messaging.
- [17] Documentation officielle de Java NIO (New I/O)
- [18] Documentation sun.misc.Unsafe
- [19] Shipilëv Aleksey. Java Microbenchmark Harness (JMH)
- [20] Stock Exchange System Design | Order Matching, Trading Engine & Scalability Explained - Youtube
- [21] How Does a Matching Engine Work - Juan D Bustamante - Devnexus 2022 - Youtube